

True Random Number Generation on the nRF52840 CryptoCell CC310

Extracting hardware entropy from an ARM CryptoCell-310 on a Sseed XIAO nRF52840, and validating it against the NIST SP 800-22 statistical test suite.

BOARD	ENTROPY SOURCE	INTERFACE	LARGEST SAMPLE
XIAO nRF52840	CryptoCell CC310	CRYS_RND	7,948,288 bits
TESTS PASSED	TRANSFER RATE		
13 / 13	~1 MB / 18 s		

SUMMARY

What was built

A complete pipeline that pulls true random numbers from the nRF52840's dedicated hardware crypto block, transfers them to a PC, and subjects them to the full NIST SP 800-22 battery. Every test passes on genuine CC310 output.

The work also produced an independent, from-scratch implementation of the SP 800-22 suite in Python, which was itself validated against known-good and known-bad reference data before being trusted on hardware output.

Why this is a *true* RNG

Ordinary software random number generators are deterministic: given the same starting seed, they replay the same sequence forever. That is fine for simulations and fatal for cryptography.

The nRF52840 carries a dedicated hardware security block, the **ARM CryptoCell-310**. Inside it, a ring-oscillator circuit harvests genuine physical noise — thermal and electrical fluctuation in the silicon itself — that no algorithm can predict or reproduce.

That raw noise feeds a **CTR_DRBG**, a deterministic generator standardised in NIST SP 800-90A, which is continuously reseeded from the physical source. This is the conventional, certified architecture: raw entropy is almost never handed out directly, because a conditioning stage removes bias while the hardware reseeding preserves genuine unpredictability.

The three-stage pipeline

01 Generate — on the microcontroller

An Arduino sketch initialises the CryptoCell and requests random bytes in 128-byte blocks via the `CRYS_RND` API.

02 Transfer — over USB serial

Bytes are hex-encoded and streamed as newline-delimited lines between `# BEGIN` and `# END` markers. A Python script decodes them back into a raw binary file.

03 Test — on the PC

The binary file is run through all thirteen SP 800-22 tests, producing a p-value and verdict for each.

Why hex instead of raw binary

Raw bytes would halve the transfer size, but a stray `0x0A` byte is indistinguishable from a newline, and any framing glitch would silently corrupt everything downstream. Hex encoding is self-delimiting: each line stands alone and decodes independently.

The initialisation order that cost days

The CC310 refused to produce output for a long stretch of this project. Every attempt behaved identically: initialisation reported success, then the first request for random bytes hung forever. No serial output, no USB enumeration, unrecoverable by reset — indistinguishable from dead silicon.

THE BUG

The CryptoCell hardware must be powered up with `SaSi_LibInit()` *before* any `CRYS_*` call. Code that constructs an `nRFCrypto_Random` and calls its `begin()` directly skips that step.

`CRYS_RndInit()` performs only software-side setup, so it returns **success regardless** — then `CRYS_RND_GenerateVector()` blocks forever, polling hardware that was never switched on.

The correct entry point does both steps in order:

```
nRFCrypto.begin()  
├─ SaSi_LibInit()    // powers up the CryptoCell hardware  
└─ CRYS_RndInit()    // then initialises the RNG engine  
  
nRFCrypto.Random.generate(buf, n)  
└─ CRYS_RND_GenerateVector()
```

The failure mode is worth recording because it mimics a hardware fault so convincingly. A silent success followed by an unrecoverable hang is a far more misleading signal than an error code would have been.

A silent corruption in the capture path

The original capture script, on encountering a truncated line, carried the leftover half-byte onto the following line. That shifted every subsequent byte by four bits.

This is the most dangerous class of bug in a project like this: shifted random data still *looks* perfectly random, so nothing visibly breaks — but the file no longer represents what the chip produced. It surfaced only as the Serial test sitting at `0.010283`, fractionally above the 0.01 pass threshold.

Truncated lines are now discarded whole and counted. The same test then returned `0.028721`, comfortably clear. Every byte in the final datasets is byte-aligned and exactly as generated.

Validating the test suite itself

A test suite that reports PASS on everything is worthless. Before trusting it on hardware output, the implementation was run against four reference datasets with known expected outcomes.

SUITE VALIDATION AGAINST REFERENCE DATA

DATASET	EXPECTED	OBSERVED
os.urandom	Pass all	3 / 3 fresh samples clean
All-zero bytes	Fail nearly all	11 / 13 failed
70%-biased bitstream	Fail nearly all	10 / 13 failed
Weak LCG	Fail spectral	8 / 13 failed, incl. DFT

The suite discriminates correctly, so its verdict on real hardware output can be trusted.

CC310 hardware output

993,536 bytes — 7,948,288 bits — of raw `CRYS_RND` output, captured in approximately 18 seconds.

NIST SP 800-22 · RAW CC310 · A = 0.01

TEST	P-VALUE	RESULT
Frequency (Monobit)	0.505782	PASS
Block Frequency	0.476197	PASS
Runs	0.127776	PASS
Longest Run of Ones	0.188890	PASS
Binary Matrix Rank	0.125504	PASS
Discrete Fourier Transform	0.919110	PASS
Non-overlapping Template	0.805184	PASS
Maurer's Universal	0.584228	PASS
Serial	0.028721	PASS
Approximate Entropy	0.882174	PASS
Cumulative Sums	0.430308	PASS
Random Excursions	0.024265	PASS
Random Excursions Variant	0.074681	PASS
Overall verdict	PASS — 13 / 13	

Independent confirmation

A separate 250,000-byte capture, taken from a fresh reset, also passed all thirteen tests — worst p-value `0.219608`.
Reproducibility across independent captures matters more than any single run.

How to read a p-value here

A p-value of **0.01 or above passes**. It is emphatically *not* a quality score: 0.02 and 0.98 are equally good results. It expresses the probability of observing data this unusual if the source were genuinely random.

Because the suite performs roughly forty individual comparisons at $\alpha = 0.01$, a perfect source still has close to a **one-in-three chance of one spurious failure per run**. That is what $\alpha = 0.01$ means, not evidence of a defect.

The correct response to a single isolated failure is to recapture and re-run. Repeated failures, or the same test failing across independent captures, is the signal that matters.

Which figures describe the hardware

Two firmware variants exist. The distinction is important when citing results.

SKETCH	STREAMS	USE FOR
cc310_capture	Raw CC310 output	Hardware evidence
cc310_rngclass	CC310 → ChaCha20 pool	Integration demo

The second variant wires the CC310 into a software entropy pool as a noise source. It works and passes all thirteen tests, but that result describes **ChaCha20, not the CryptoCell**: a software stream cipher passes SP 800-22 even when fed a poor entropy source, so it cannot serve as evidence about hardware quality.

All hardware figures in this report come from raw `CRYS_RND` output.

Running it yourself

Flash `cc310_capture.ino` to the board, then:

```
pip install pyserial numpy scipy
```

```
python capture_trng.py --port COM16 --out random.bin --bytes 1000000
```

```
python sp800_22.py random.bin
```

Source, datasets and full results: github.com/SudeepRD001/TRNG